Sistema de Estacionamento

Documentação do CRUD de Proprietários e Veículos

Disciplina: Programação para Dispositivos Móveis II

Tecnologias: React Native, Node.js, Express, Sequelize e MySQL

Kaique Nascimento de Oliveira

DSM-5

29/10/2026

1. Descrição do projeto e tecnologias utilizadas

O projeto é um sistema de estacionamento que permite cadastrar proprietários e veículos e associar cada veículo a um proprietário. É dividido em um Back-End (API REST em Node.js) e um Front-End mobile (React Native) que consome essa API. Cada entidade possui as quatro operações do CRUD: consultar (GET), cadastrar (POST), alterar (PUT) e excluir (DELETE).

Banco de dados estacionamento:

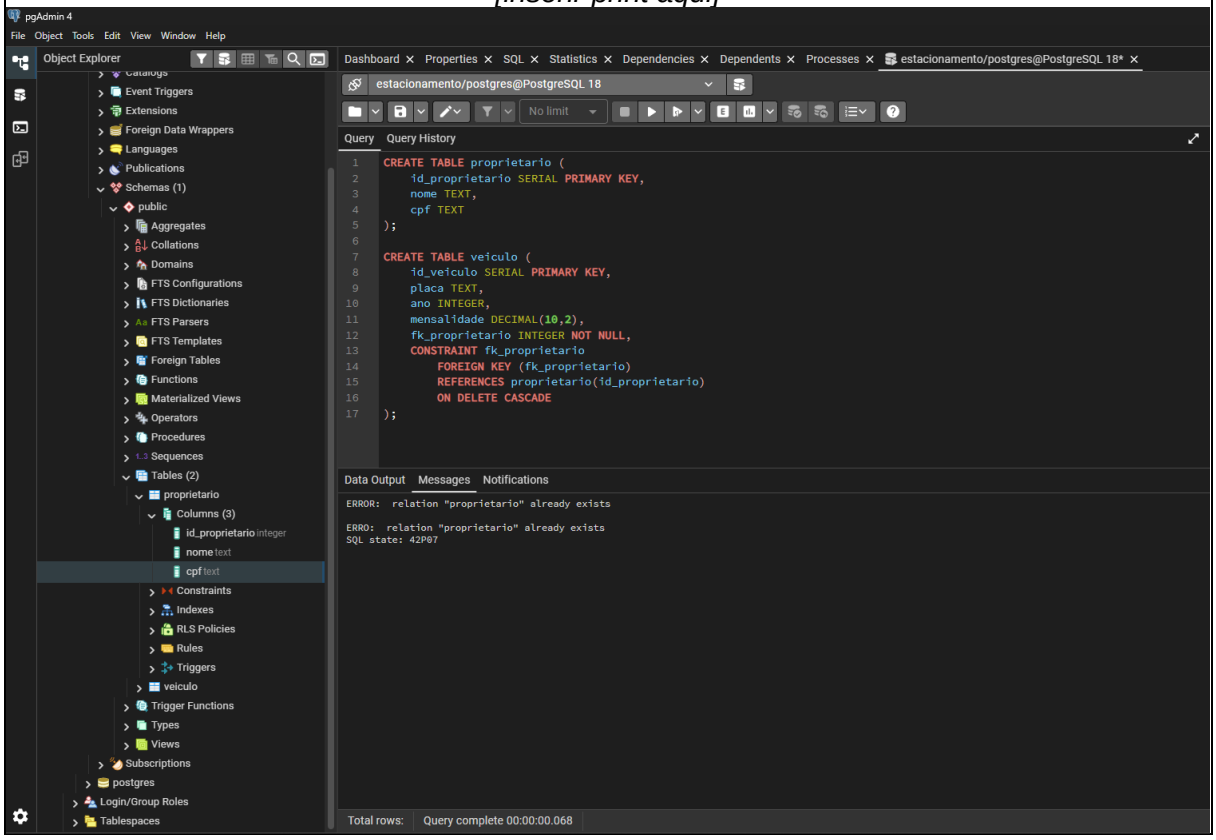
- proprietario: id_proprietario (chave primária), nome e cpf.
- veiculo: id_veiculo (chave primária), placa, ano, mensalidade e fk_proprietario (chave estrangeira para proprietario, com exclusão em cascata).

Tecnologias:

- MySQL: banco de dados relacional.
- Node.js e Express: servidor e rotas da API.
- Sequelize e mysql2: ORM e driver de conexão com o MySQL.
- body-parser e cors: leitura do JSON das requisições e liberação de acesso à API.
- React Native (Expo e Expo Router) com TypeScript: aplicativo mobile, telas e navegação.
- Axios: requisições HTTP do aplicativo para a API.
- @react-native-picker/picker: seleção do proprietário no cadastro de veículo.

Banco de dados criado

[inserir print aqui]



2. Estrutura de pastas do Front-End e do Back-End

2.1 Back-End

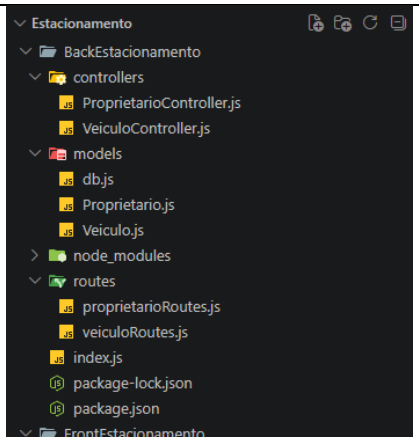
```
BackEstacionamento/
├── controllers/
│   ├── ProprietarioController.js
│   └── VeiculoController.js
├── models/
│   ├── db.js
│   ├── Proprietario.js
│   └── Veiculo.js
├── index.js
└── package.json
```

2.2 Front-End

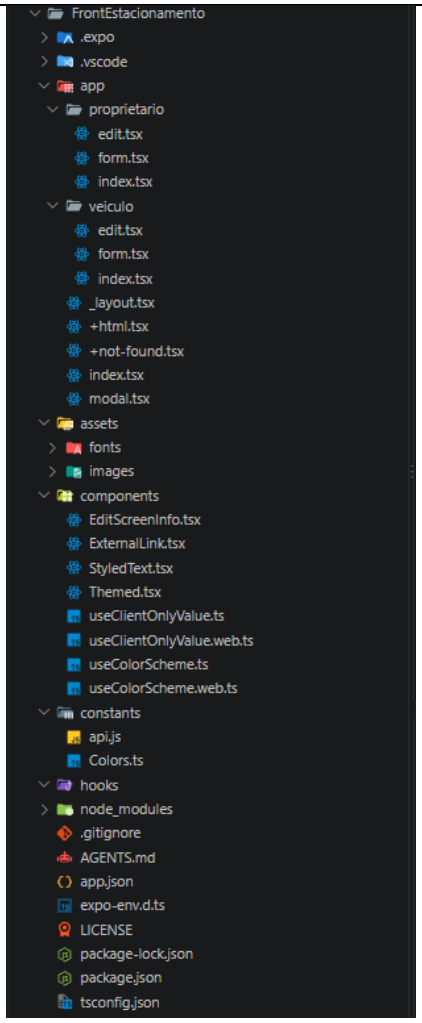
```
FrontEstacionamento/
├── app/
│   ├── _layout.tsx
│   ├── index.tsx
│   ├── proprietario/  (index.tsx, form.tsx, edit.tsx)
│   └── veiculo/       (index.tsx, form.tsx, edit.tsx)
├── components/
│   ├── AppButton.tsx
│   ├── FormField.tsx
│   ├── ListCard.tsx
│   ├── OwnerPicker.tsx
│   └── HeaderAdd.tsx
├── constants/
```

```
│   ├── api.ts
│   ├── colors.ts
│   └── styles.ts
├── hooks/
│   ├── types.ts
│   ├── useProprietarios.ts
│   └── useVeiculos.ts
└── package.json
```

Estrutura de pastas



BackEstacionamento



FrontEstacionamento

3. Códigos organizados e explicação de cada arquivo

O código-fonte completo está no projeto compactado (ZIP) entregue junto com este documento. Abaixo está a função de cada arquivo.

3.1 Back-End

Arquivo	Função
models/db.js	Configura a conexão do Sequelize com o banco estacionamento (usuário, senha, host e dialeto MySQL).
models/Proprietario.js	Define a tabela proprietario (id_proprietario, nome, cpf). O sync é executado uma vez para criar a tabela e depois comentado.
models/Veiculo.js	Define a tabela veiculo (id_veiculo, placa, ano, mensalidade) e a chave estrangeira fk_proprietario, com exclusão em cascata.
controllers/ProprietarioController.js	Rotas do CRUD de proprietários: GET /, GET /:id, POST /, PUT /:id e DELETE /:id.
controllers/VeiculoController.js	Rotas do CRUD de veículos, com as mesmas operações; o corpo inclui fk_proprietario.
index.js	Cria o servidor Express na porta 8081, ativa body-parser e cors e registra as rotas /proprietario e /veiculo.

3.2 Front-End

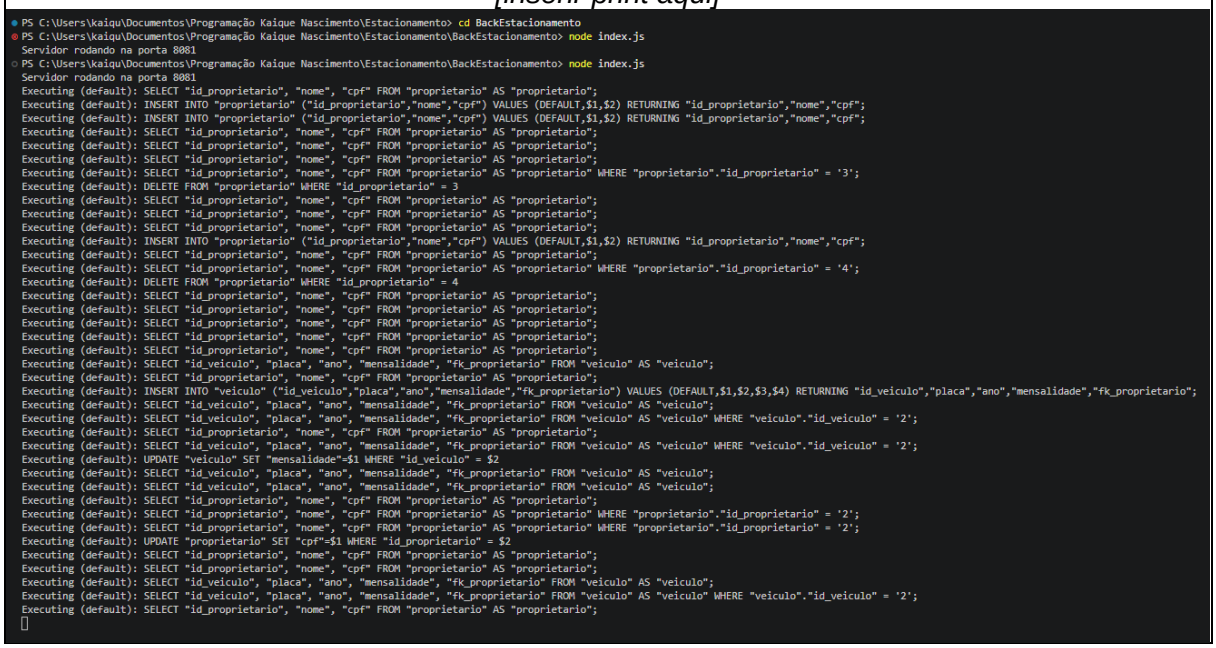
Arquivo	Função
constants/api.ts	Endereço da API em um único lugar.
constants/colors.ts	Paleta de cores do aplicativo.
constants/styles.ts	Estilos compartilhados pelas telas de proprietário e veículo.
hooks/types.ts	Tipos TypeScript de Proprietario e Veiculo.
hooks/useProprietarios.ts	Concentra as chamadas Axios de proprietários: consultar, cadastrar, alterar e excluir.
hooks/useVeiculos.ts	Concentra as chamadas Axios de veículos: consultar, cadastrar, alterar e excluir.
components/AppButton.tsx	Botão reutilizável em três cores (principal, Editar e Excluir).
components/FormField.tsx	Rótulo mais campo de texto reutilizável.
components/ListCard.tsx	Cartão das listas, com os botões Editar e Excluir.
components/OwnerPicker.tsx	Lista de seleção que associa o veículo a um proprietário.
components/HeaderAdd.tsx	Botão "+" do cabeçalho das listas.
app/_layout.tsx	Navegação em pilha e títulos das telas.
app/index.tsx	Menu inicial com acesso a Proprietários e Veículos.
app/proprietario/index.tsx	Lista de proprietários, com exclusão e acesso à edição.
app/proprietario/form.tsx	Formulário de cadastro de proprietário.
app/proprietario/edit.tsx	Formulário de edição de proprietário.

app/veiculo/index.tsx	Lista de veículos, mostrando o proprietário associado.
app/veiculo/form.tsx	Formulário de cadastro de veículo, com seleção do proprietário.
app/veiculo/edit.tsx	Formulário de edição de veículo, permitindo trocar o proprietário.

Servidor rodando

Aqui está o servidor rodando com algumas alterações que fiz para testes.

[inserir print aqui]



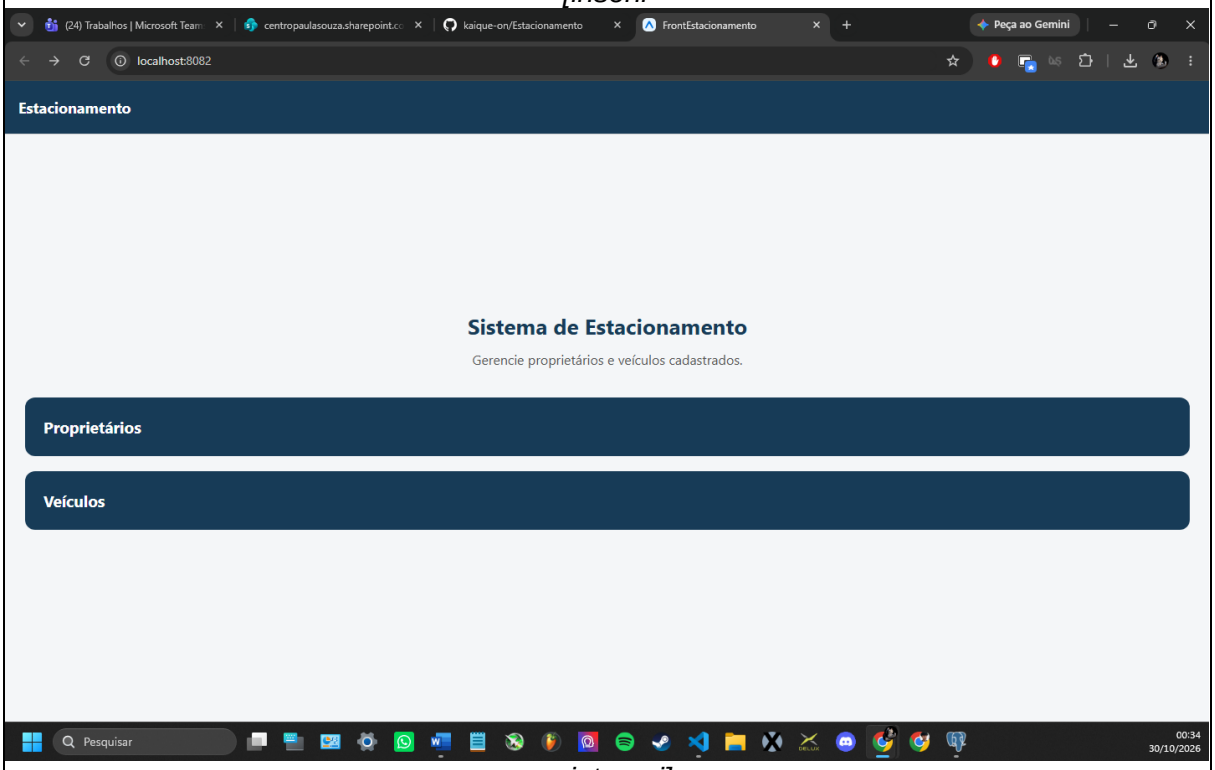
4. Prints das telas e dos testes realizados

4.1 Telas do aplicativo

Menu inicial

Tela Estacionamento com os botões Proprietários e Veículos.

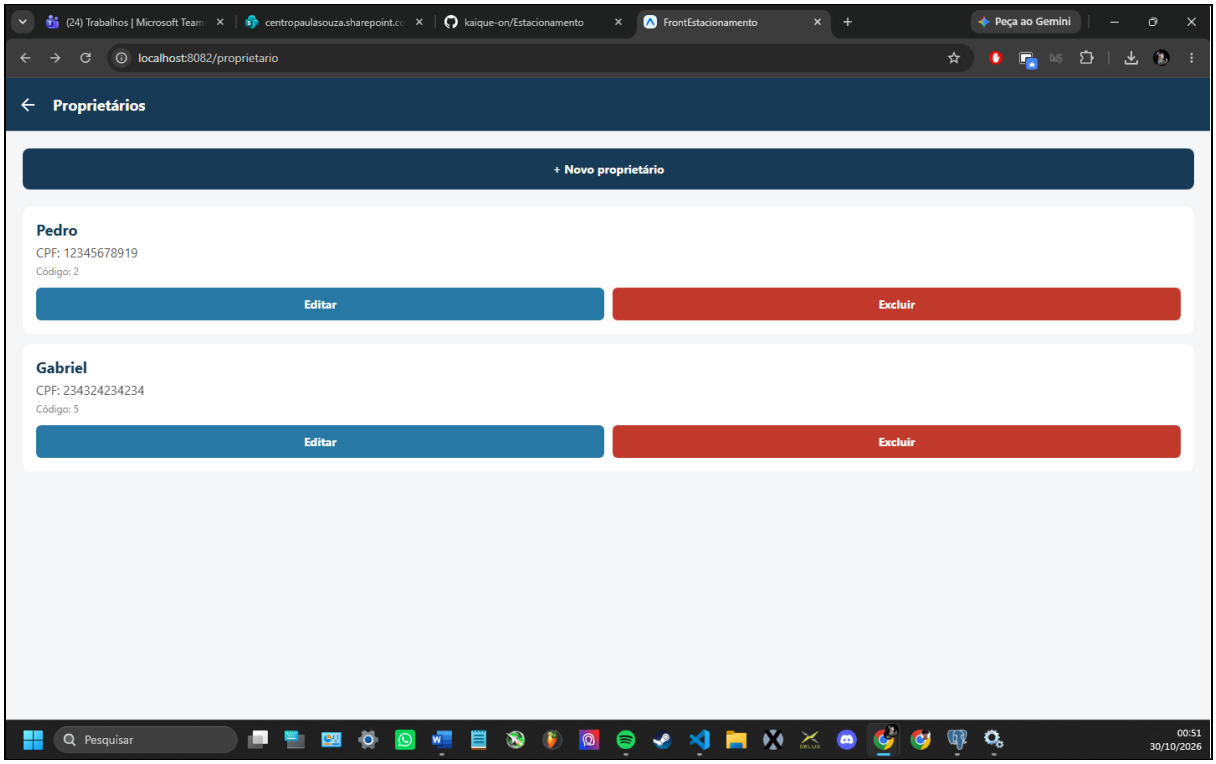
[inserir



print aqui]

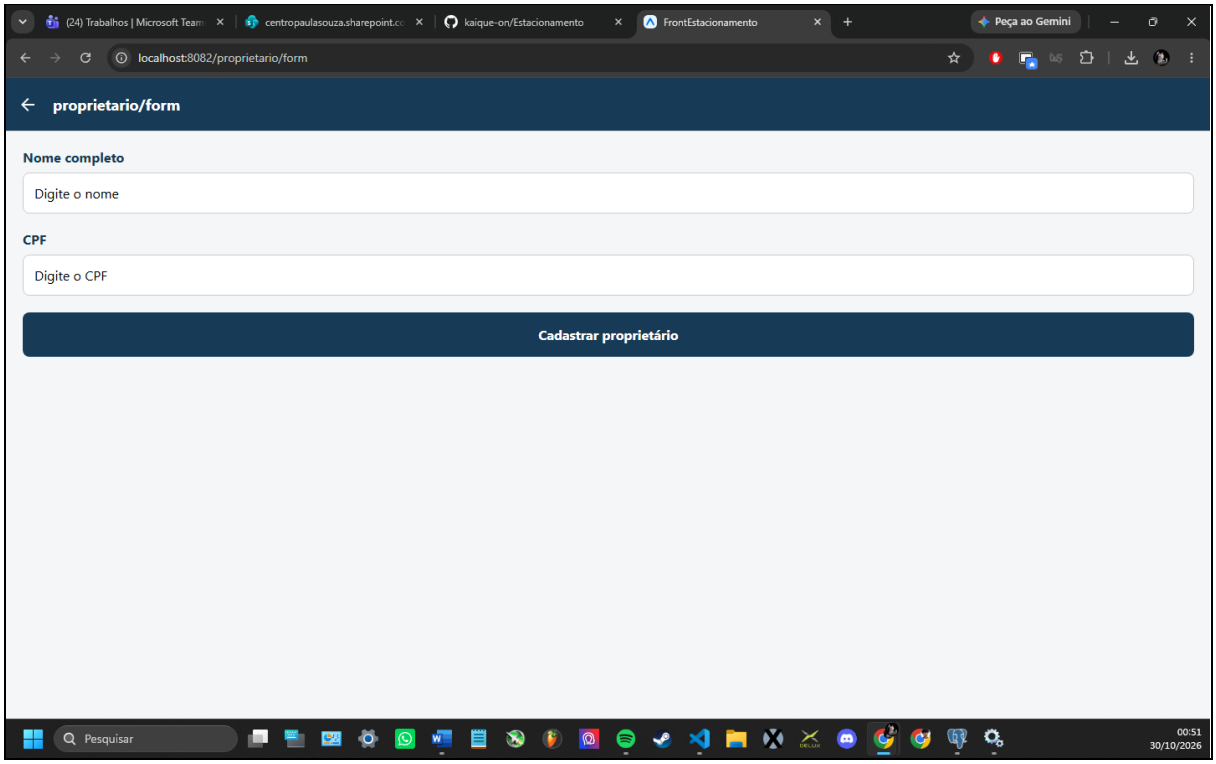
Print 5 - Lista de proprietários

Lista com pelo menos dois proprietários cadastrados.



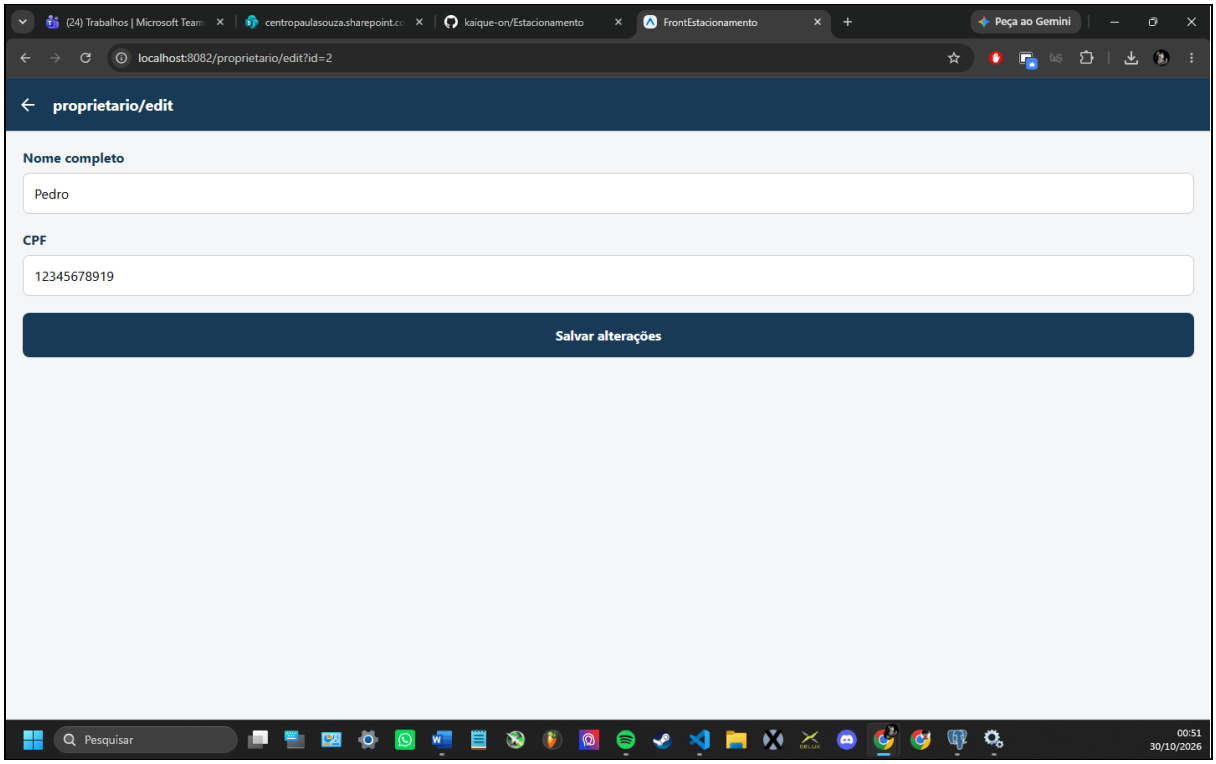
Print 6 - Cadastro de proprietário

Formulário preenchido antes de salvar.



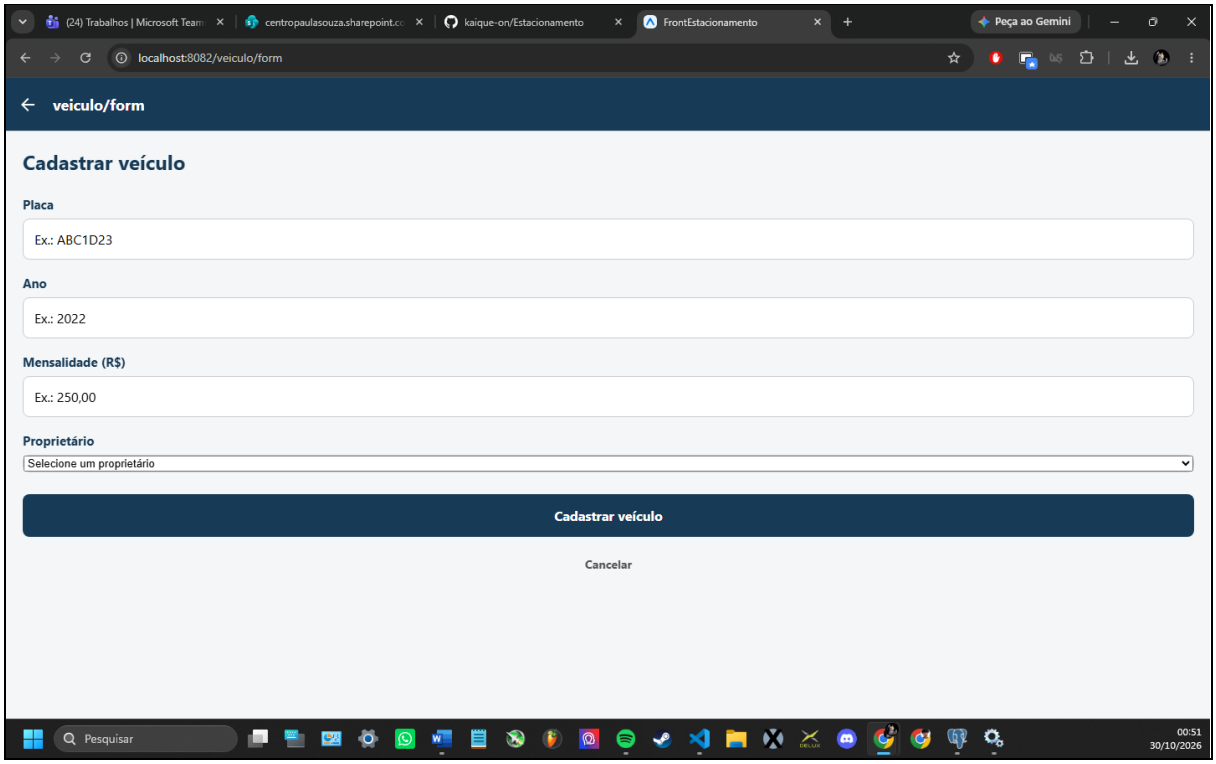
Print 7 - Edição de proprietário

Tela de edição com os dados carregados.



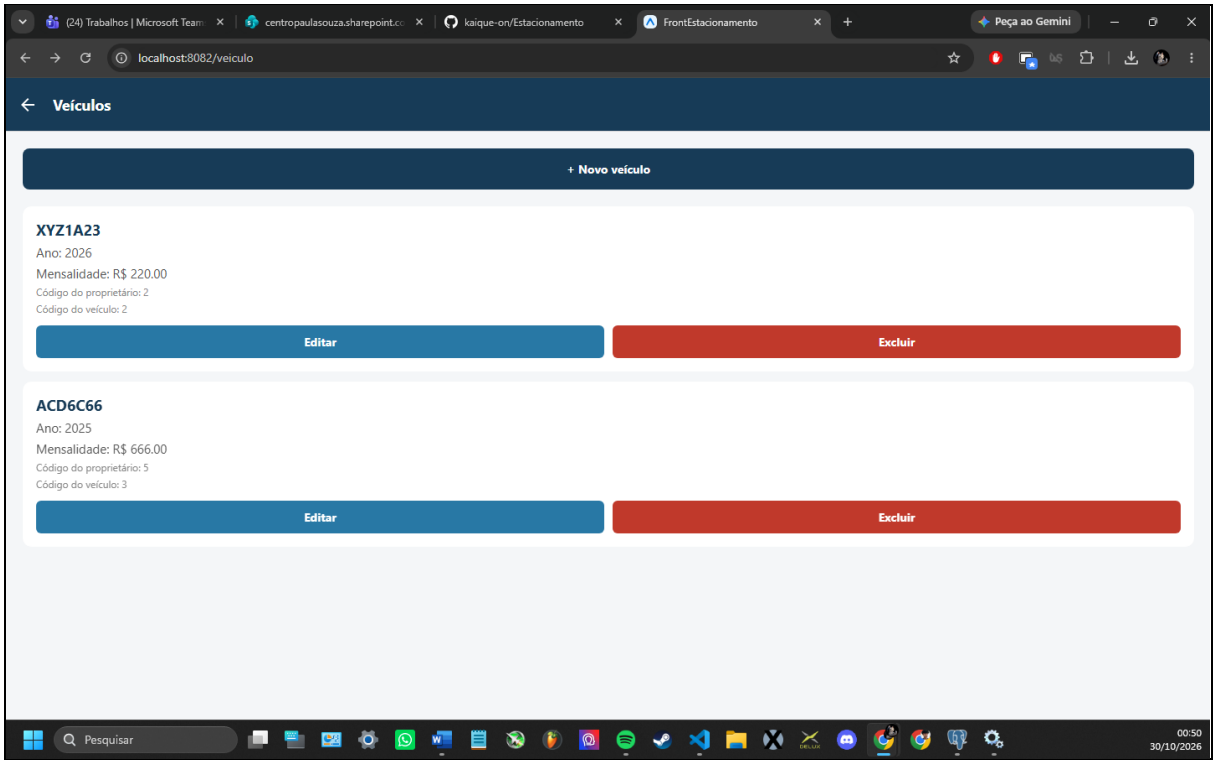
Print 8 - Cadastro de veículo

Formulário com o proprietário selecionado na lista de seleção.



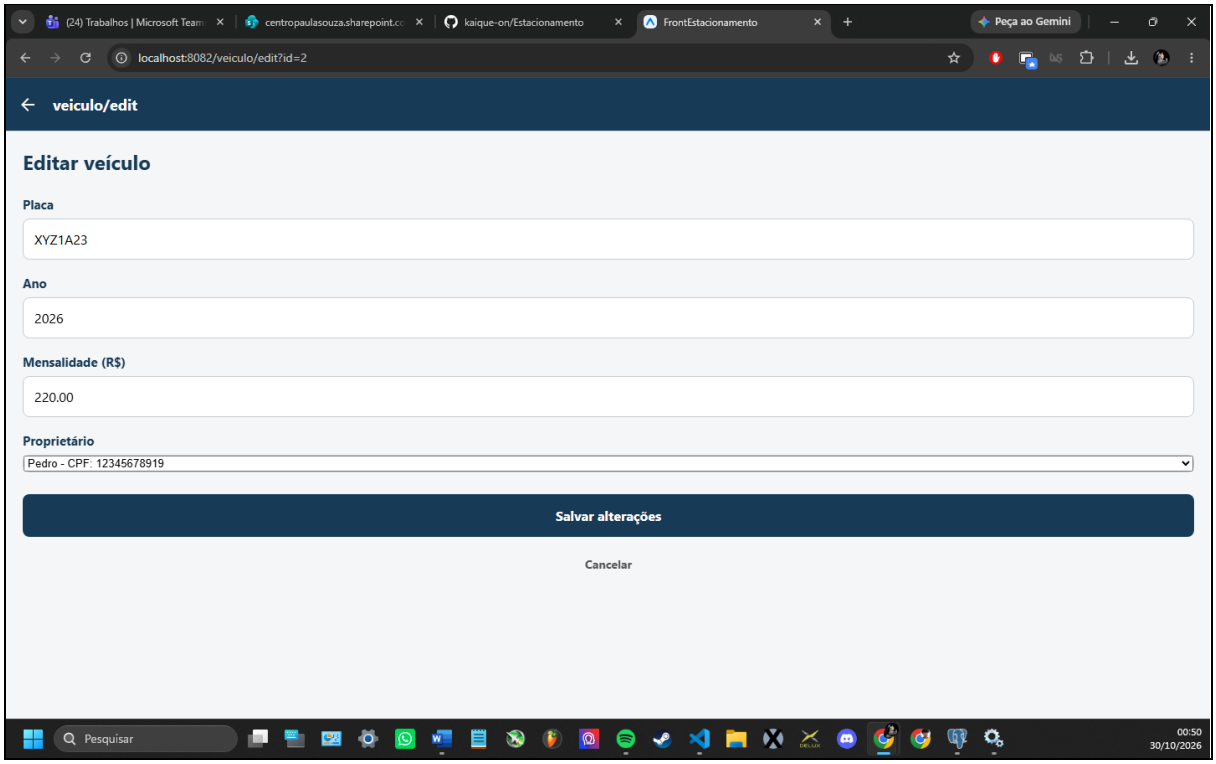
Print 9 - Lista de veículos

Lista mostrando o veículo e o proprietário associado.



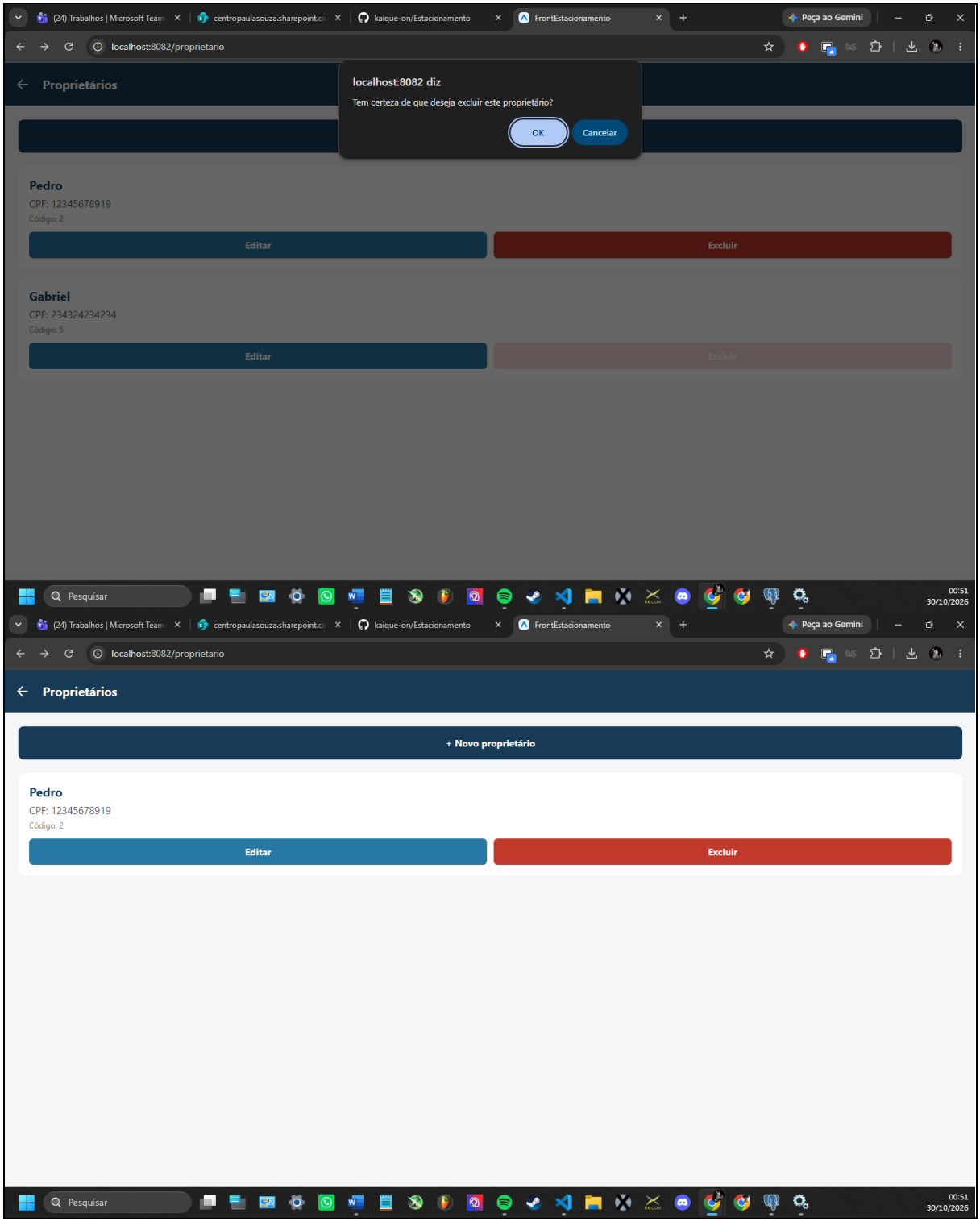
Print 10 - Edição de veículo

Tela de edição com o proprietário trocado.



Print 11 - Exclusão

Lista antes e depois de excluir um registro.



4.2 Testes do CRUD na API

Testes feitos com o servidor ativo, usando Postman ou Insomnia.

Operação	**Requisição**	**Resposta esperada**
Cadastrar proprietário	POST /proprietario	200 - Cadastrado com sucesso
Listar proprietários	GET /proprietario	200 - lista em JSON
Alterar proprietário	PUT /proprietario/:id	200 - Atualizado com sucesso
Excluir proprietário	DELETE /proprietario/:id	200 - Excluído com sucesso
Cadastrar veículo	POST /veiculo	200 - Cadastrado com sucesso
Listar veículos	GET /veiculo	200 - lista em JSON
Alterar veículo	PUT /veiculo/:id	200 - Atualizado com sucesso
Excluir veículo	DELETE /veiculo/:id	200 - Excluído com sucesso

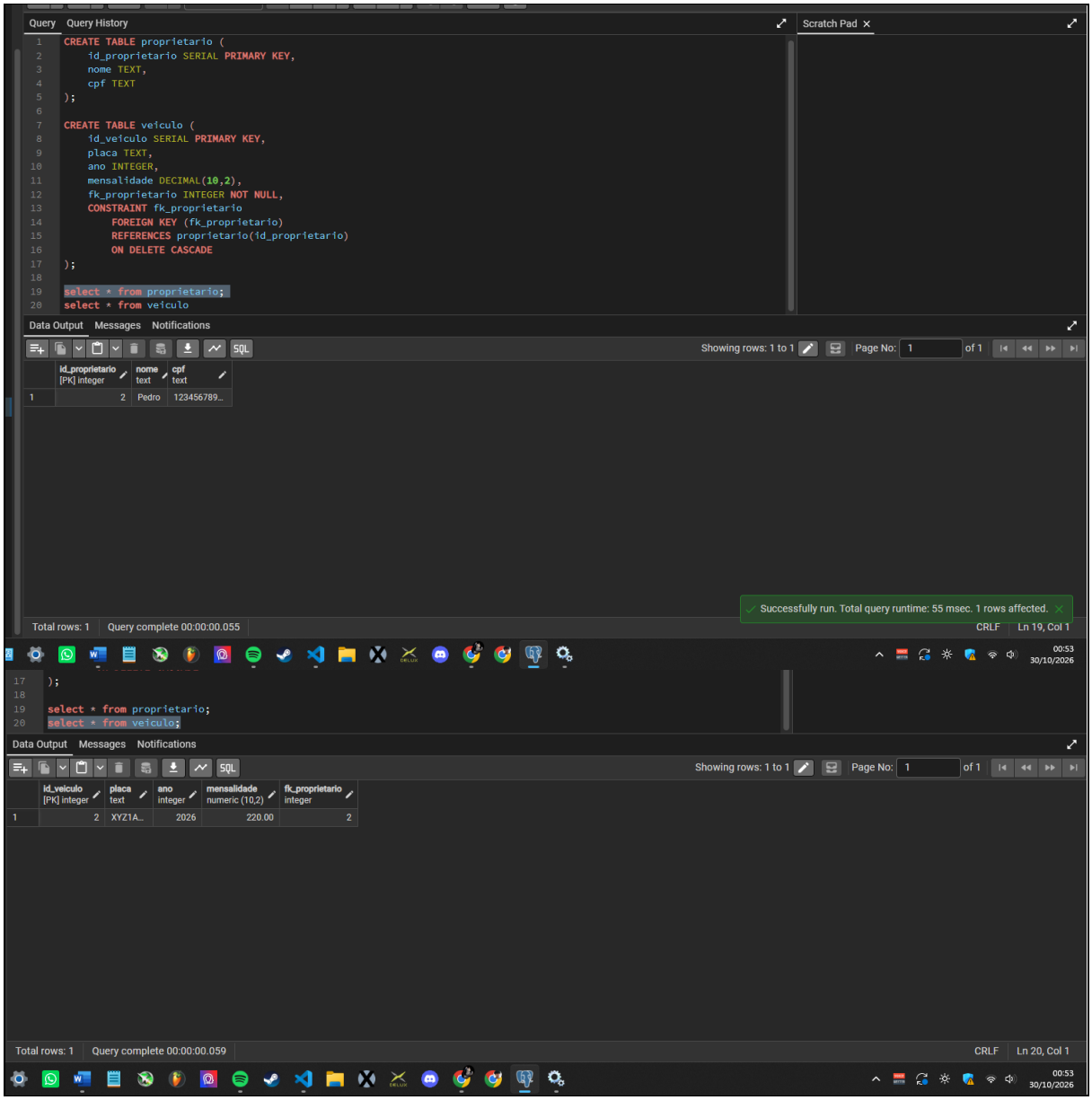
4.3 Teste completo e conferência no banco

Roteiro executado no aplicativo, conferindo cada etapa no MySQL:

1. Cadastrar um proprietário e ver na lista.
2. Editar o proprietário.
3. Cadastrar um veículo escolhendo o proprietário.
4. Editar o veículo, trocando o proprietário.
5. Excluir o veículo.
6. Excluir o proprietário e confirmar que seus veículos também foram removidos (cascata).

Queries de Banco

select * from proprietario; e select * from veiculo; mostrando os registros.



5. Comunicação entre o aplicativo, a API e o banco de dados

O sistema funciona em três camadas. O aplicativo React Native envia requisições HTTP em JSON, usando o Axios, para a API Node.js/Express. A API usa os Models do Sequelize, que transformam as chamadas em comandos SQL executados no MySQL. A resposta faz o caminho inverso até a tela.

```
Aplicativo (telas) -> hook (Axios) -> API Express (controller)
                                            |
                                      Model Sequelize
                                            |
                                  Banco MySQL (estacionamento)
```

5.1 Exemplo: cadastro de um veículo

1. Na tela de cadastro, o usuário preenche placa, ano e mensalidade e escolhe o proprietário na lista de seleção.
2. Ao tocar em Salvar, a tela chama a função de cadastro do hook useVeiculos.
3. O hook envia um POST para /veiculo, usando o endereço definido em constants/api.ts, com os dados em JSON.
4. No Back-End, o index.js encaminha a rota ao VeiculoController (o body-parser lê o JSON e o cors permite a origem do aplicativo).
5. O controller chama o Model Veiculo, e o Sequelize executa o INSERT na tabela veiculo, validando a chave estrangeira.
6. A API responde 200 com "Cadastrado com sucesso".
7. O aplicativo volta para a lista, que faz um novo GET /veiculo e mostra o registro.

5.2 Associação entre veículo e proprietário

A associação é feita pela chave estrangeira fk_proprietario da tabela veiculo, que referencia id_proprietario da tabela proprietario. O aplicativo carrega os proprietários pela API, o usuário escolhe um na lista de seleção e o id escolhido é gravado em fk_proprietario. Como a chave tem exclusão em cascata, excluir um proprietário exclui também os seus veículos.

5.3 Rotas da API (porta 8081)

Operação	**Rota**	**Comando Sequelize**
Consultar todos	GET /proprietario | /veiculo	findAll()
Consultar por id	GET /proprietario/:id | /veiculo/:id	findByPk(id)
Cadastrar	POST /proprietario | /veiculo	create({...})
Alterar	PUT /proprietario/:id | /veiculo/:id	update({...}, { where })
Excluir	DELETE /proprietario/:id | /veiculo/:id	destroy({ where })